MAE5920 · INTRODUÇÃO A DEEP LEARNING

O que a CNN lê em uma resenha?

Classificação de sentimento e tokenização no IMDb

Nestor, Letícia e Gabriel

Resumo

No teste oficial de 25.000 resenhas, TF-IDF, TextCNN e BERT ajustado atingiram 90,14%, 86,78% e 90,94% de acurácia. Em uma auditoria separada, BPE reduziu a taxa de tokens desconhecidos de 3,27% para zero, com acurácia média de 81,41% ante 82,00% com palavras. Em um par sintético que troca elogio e crítica, as seis CNNs seguiram a polaridade da primeira oração. Em outro par, as mesmas janelas em ordem diferente produziram logits iguais. Guardar dois picos na ordem do texto distinguiu os escores desse controle, com acerto semelhante nos pares editados por pessoas.

1 Introdução

Classificamos resenhas de filmes do IMDb como positivas ou negativas. Comparamos TF-IDF com regressão logística, uma TextCNN treinada do zero e BERT ajustado. O objetivo didático é entender como tokenização, janelas de convolução e máximo global condicionam as decisões da CNN. Além das métricas médias, usamos alterações controladas de texto e exemplos de outros domínios para investigar seus limites.

Duas frases, dois finais

As micro-resenhas abaixo usam as mesmas palavras. A troca coloca um elogio ou uma crítica depois de *but*, onde lemos o veredito.

A / leitura positiva

*I thought this movie was terrible, but it was **wonderful**.*

Termina com elogio.

B / leitura negativa

*I thought this movie was wonderful, but it was **terrible**.*

Termina com crítica.

Cada palavra aparece o mesmo número de vezes. Um modelo que só conta palavras isoladas recebe a mesma entrada. O TF-IDF do estudo inclui bigramas, que podem mudar com a troca. A CNN também recebe informação de ordem dentro de cada janela. A Seção 5 mostra o que cada treino fez com os dois finais.

Uso de ferramentas de IA

Usamos o Codex, da OpenAI, na implementação de código, execução e análise dos experimentos e elaboração e revisão de textos, figuras e slides. O Qwen3-8B foi empregado como modelo experimental. Os resultados quantitativos provêm das execuções registradas no repositório. A revisão final e a responsabilidade pelo conteúdo cabem aos autores.

Código, notebook executado e métricas por execução. Repositório do projeto.

2 Dados e protocolo de avaliação

O IMDb tem 25.000 resenhas em inglês no treino oficial e 25.000 no teste [1], com rótulos 0 (negativo) e 1 (positivo). A limpeza decodifica HTML, remove tags e normaliza espaços em cada texto.

Papéis das amostras

Papel	Original	Ensaios internos	Uso
Treino	19.823	6.144	Ajustar representação, vocabulário e pesos.
Validação	2.478	1.536	Escolher hiperparâmetros e época.
Calibração	2.478	–	Ajustar temperatura após a seleção original.
Teste/auditoria	25.000	2.048	Avaliar a configuração já escolhida.

Tabela 1. Cada extensão tem auditoria própria, aberta após congelar pesos e escolhas.

Os ensaios internos com sequências usam até 128 palavras. Na auditoria BPE, esse teto cortou 1.535 resenhas e manteve 51,24% dos tokens por palavras. Os prefixos já cabiam no teto adicional de 256 tokens. Protocolo sequencial e protocolo BPE.

Comprimento antes dos recortes

Unidade	n	Mín.	P10	Média	Mediana	P90	Máx.
Resenha no treino	19.823	10	90	231,1	173	451	2.459
Resenha no teste	25.000	4	88	226,2	170	439	2.235
Frase no teste*	312.557	1	5	18,1	16	33	722

Tabela 2. Palavras separadas por espaços no texto limpo. *Frases estimadas por pontuação final. Distribuições completas.

Análise de possíveis vazamentos de informação

Removemos 123 linhas do treino com hash presente no teste e 98 duplicatas internas, usando SHA-256 do texto limpo com *casefold*. Para excluir duplicatas, lemos o texto do teste e reservamos os rótulos à avaliação. Os quatro conjuntos finais têm zero hashes em comum. O teste conserva 201 ocorrências duplicadas. Manifesto.

Ajustamos TF-IDF, vocabulário da CNN e fusões do BPE no treino. A checagem cobre igualdade de texto, e paráfrases ou exposição no pré-treino seguem em aberto. A análise de erros e o Qwen reutilizam o teste em caráter exploratório.

Dicionário de dados resumido

Campo	Significado
`id`	Identificador derivado da divisão oficial e da posição na fonte.
`label`	Classe binária usada na função de perda e na avaliação.
`clean_text`	Texto após decodificação de HTML e normalização de espaços.
`word_count`	Número de segmentos separados por espaço no texto limpo.
`text_sha256`	Impressão do texto normalizado usada para verificar duplicações.

Tabela 3. A única entrada do classificador é o texto. Dicionário completo.

Métricas

Acurácia é a fração de acertos. F1 macro dá o mesmo peso ao F1 de cada classe. A entropia cruzada (CE) é a média de $-\log p(\text{classe verdadeira})$.

3 Arquitetura da TextCNN

A TextCNN usada nos ensaios por palavras e por BPE reúne 300 filtros [2, 4]. Cada filtro entrega a maior resposta entre suas janelas. Filtros diferentes podem atingir o máximo em trechos diferentes. A camada final aprende quanto cada uma das 300 respostas conta para cada classe.

Tokenização por regex e por BPE

A mesma expressão regular, ou *regex*, separa palavras, contrações e números em minúsculas. Na CNN por palavras, essas unidades vão direto ao vocabulário dos 19.998 termos mais frequentes, mais `<pad>` e `<unk>`. Na versão BPE, enviamos as palavras ainda como texto ao tokenizador, que aplica fusões aprendidas nos 6.144 textos de treino, com frequência mínima 2 e sem rótulos [3]. As duas CNNs são treinadas separadamente e têm vocabulários de 20.000 IDs. `</w>` marca o fim da palavra. Normalização e vocabulários.

Quando uma palavra vira três peças *The pallbearer scene was unexpectedly moving.*

Por palavras `<unk>` BPE `p` `all` `bearer</w>`

Uma janela de largura 3 pode cobrir as três peças de uma única palavra.

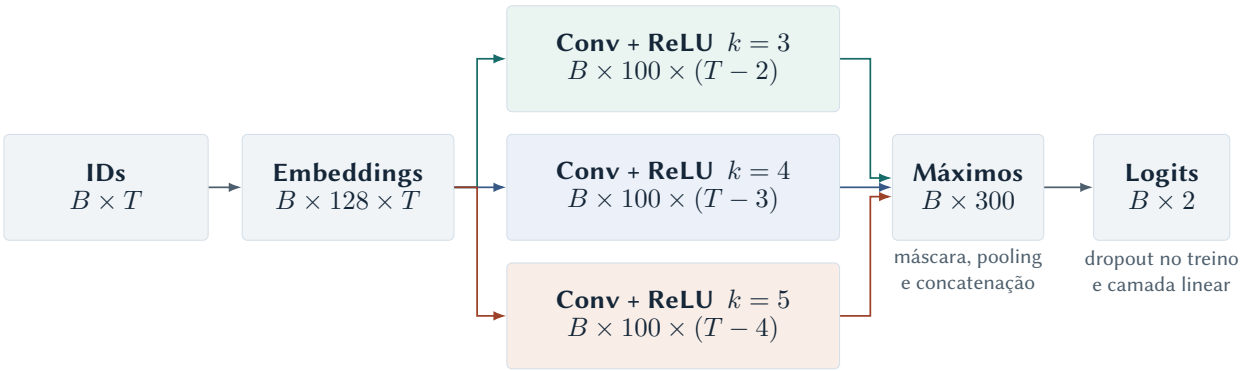


Figura 1. Fluxo da TextCNN. B é o número de resenhas no minibatch e T é o comprimento com padding. As dimensões representam tensores. Os trechos abaixo levam às linhas da implementação.

A mesma conta em cada janela

A tabela de embeddings tem 20.000×128 valores. Um lote de $B \times T$ IDs gera $B \times T \times 128$. A transposição põe os 128 canais antes de T. Cada filtro de largura k lê k tokens consecutivos em ordem. Usamos larguras 3, 4 e 5. O filtro f reaplica os pesos W_f e o viés b_f em cada janela. Se e_t é o embedding na posição t, temos

$$a_{f,t} = \max(0,\ \langle W_f, [e_t, \ldots, e_{t+k-1}] \rangle + b_f)\,, \qquad m_f = \max_{t \in \mathcal{V}_k} a_{f,t}.$$

A ReLU põe respostas negativas em zero. O máximo m_f guarda a altura do pico e descarta sua posição. Para uma sequência de L tokens, a máscara mantém $\max(1, L - k + 1)$ posições iniciais. Se $L < k$, fica uma janela com padding. `embedding`, `conv`, `masked_fill`.

Do pico à classe

Os máximos formam um vetor m de 300 componentes. No treino, passa-se por dropout de 0,5 antes da camada linear. Na inferência, a saída é $z = Wm + b$, com dois logits, z_- e z_+. A margem $\Delta = z_+ - z_-$ expressa a diferença entre os votos das duas classes. O softmax binário dá $p(+) = \sigma(\Delta)$. Uma margem positiva corresponde a $p(+) > 0{,}5$. Na frase A da semente 17 por palavras, $z_- \approx 2{,}259$ e $z_+ \approx -2{,}863$ dão $p(+) \approx 0{,}59\%$. A entropia cruzada do treino recebe os dois logits diretamente. `classifier`.

Onde estão os 2.714.502 parâmetros da CNN por palavras?

$20.000 \times 128 = 2.560.000$ nos embeddings. $100 \sum_{k \in \{3,4,5\}} (128k + 1) = 153.900$ nas convoluções. $2(300 + 1) = 602$ na saída.

4 Experimentos e resultados

Otimização e seleção de época

Usamos os gradientes da entropia cruzada para atualizar embeddings, filtros e saída, com norma global limitada a 1. Nos ensaios internos, inicializamos embeddings com desvio 0,1 e padding zero, filtros com He e saída com Xavier [4].

Otimizador	Taxas testadas	Escolhida
SGD	0,001 0,03 0,3	0,3
SGD com momentum 0,9	0,001 0,01 0,1	0,1
Adam	0,0003 0,001 0,003	0,001

Tabela 4. Três sementes por célula (17, 29 e 43), minibatches de 64 e orçamento de 12 épocas. A CNN base tem larguras 3/4/5, cem filtros por largura e dropout de 0,5.

Na mesma semente, fixamos pesos iniciais e ordem dos lotes. Guardamos a época de menor CE de validação e escolhemos a taxa pela média dessa perda. A época zero é a inicialização. Os ensaios de CNN usam dropout e seleção de época, com decaimento de pesos zero. A CNN original usa AdamW com *weight decay* de 0,01, dropout de 0,5 e seleção por F1 macro de validação, com paciência de duas épocas. O clipping limita o passo causado por gradientes grandes.

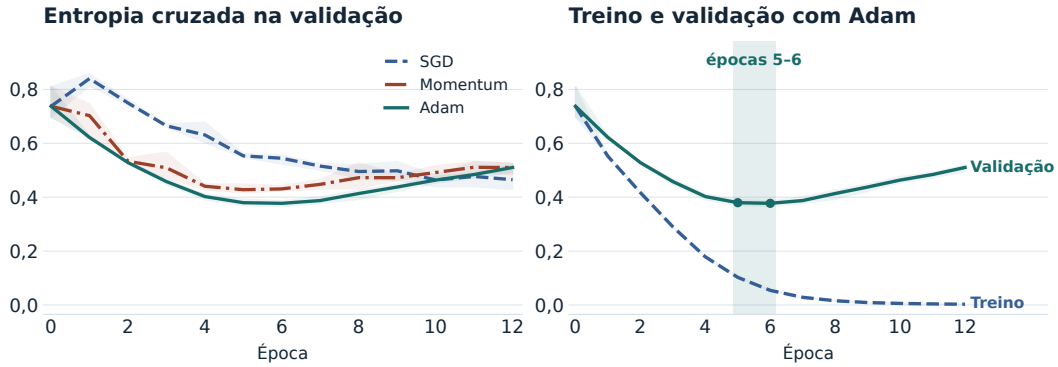


Figura 2. As linhas são médias e as faixas cobrem mínimo e máximo entre sementes. À esquerda, cada otimizador usa sua taxa escolhida. À direita, a perda de treino com Adam continua caindo depois da melhor região de validação. Os checkpoints de Adam vêm das épocas 5 e 6.

Largura, número de filtros e dropout

Na auditoria, as larguras 3 e 5, com 100 filtros, deram 81,27% e 80,94%. Com 3/4/5, 300 filtros deram 81,95% e 96 deram 81,64%. Com dropout de 0,8, a acurácia foi 80,44%. Métricas por semente. Código dos treinos.

Regex e BPE no mesmo experimento

As duas CNNs recebem o mesmo prefixo de até 128 palavras. Usam Adam a 0,001, as mesmas dimensões, dropout, minibatches e orçamento. Escolhemos a época na validação. Na mesma semente, a matriz inicial é igual, e suas linhas representam unidades diferentes.

Tokenizer	Acurácia	F1 macro	CE	UNK	Tokens médios
Regex por palavras	82,00%	0,8200	0,3964	3,27%	121,3
BPE 20 mil	81,41%	0,8140	0,4052	0,00%	129,1

Tabela 5. Médias de três sementes em 2.048 resenhas. UNK é a fração de tokens desconhecidos. BPE reduziu UNK, alongou as sequências e caiu 0,59 ponto em acurácia. Métricas por execução.

TF-IDF e regressão logística

Em *not good*, contamos *not*, *good* e o par *not good*. Os bigramas captam ordem local. O vetor agrega as ocorrências da resenha inteira e perde suas posições. TF mede a frequência local e IDF reduz o peso de termos comuns no treino. A regressão usa os valores normalizados x_j, aprende os coeficientes w_j com os rótulos e aplica a sigmoide à soma $b + \sum_j w_j x_j$. Código do baseline.

Resultados no teste oficial do IMDb

O TF-IDF lê a resenha inteira. A CNN usa até 400 tokens por regex e o BERT até 192 WordPieces [5]. Ao reajustar o TF-IDF com o mesmo recorte da CNN, a acurácia caiu de 90,15% para 89,66% no controle. No ajuste original com L2 ($C = 8$), o baseline atingiu 99,72% no treino. Penalização mais forte reduziu o F1 de validação. Na busca ampliada, $C = 16$ venceu na validação e fez 90,04% no teste. Com rótulos embaralhados, o modelo atingiu 99,75% no treino e 50,08% na validação. Controle de capacidade. Busca ampliada. Recorte.

Divisão	Modelo	Acurácia	F1 macro	CE bruta
Treino	TF-IDF + logística	99,72%	0,9972	0,0973
Treino	CNN	94,95%	0,9494	0,1567
Treino	BERT	96,32%	0,9632	0,1138
Validação	TF-IDF + logística	91,53%	0,9152	0,2255
Validação	CNN	87,97%	0,8796	0,2877
Validação	BERT	91,28%	0,9128	0,2404
Teste	TF-IDF + logística	90,14%	0,9014	0,2531
Teste	CNN	86,78%	0,8678	0,3040
Teste	BERT	90,94%	0,9094	0,2433

Tabela 6. Escores antes da calibração. Treino com 19.823 resenhas, validação com 2.478 e teste oficial com 25.000. Resultados originais.

Discordâncias entre TF-IDF e CNN e efeito do recorte

Cruzamos as previsões da CNN e do TF-IDF pelo ID das 25.000 resenhas. Apenas o TF-IDF acertou 1.715 e apenas a CNN acertou 875. O teto de 400 limita padding e cálculo. O valor vem da configuração inicial, sem busca documentada. Cortou 3.118 resenhas (12,47%) e 10,41% dos tokens. Com os mesmos pesos, o texto inteiro elevou a acurácia de 82,33% a 86,31% nas resenhas cortadas e de 86,78% a 87,28% no teste. Comparação pareada.

Resenha longa / 780 tokens / rótulo negativo

A frase "*I had to leave because it was so bad.*" aparece depois do limite de 400 tokens. Passamos o prefixo e o texto inteiro aos mesmos modelos. Com a entrada maior, a CNN percorre mais janelas com os mesmos filtros.

	TF-IDF $p(+)$	CNN $p(+)$
Primeiros 400 tokens	71,7%	94,7%
Texto inteiro	6,5%	12,1%

Com limiar de 50%, a classe dos dois muda de positiva para negativa. No texto inteiro, janelas como *hate to waste* empurram a margem da CNN para o negativo.

Resenha curta / 61 tokens / rótulo positivo

Ambos leem o texto inteiro. A CNN dá 96,6% de escore positivo e o TF-IDF, 45,5%. Na CNN, *part is brilliant* soma $+0{,}257$ à margem e *the ending doesn't make much* soma $-0{,}167$. No TF-IDF, *brilliant* soma $+0{,}503$ e o bigrama *not one* soma $-0{,}337$, embora esse bigrama esteja numa comparação de enredos.

Escores com temperaturas originais. Contribuições à margem bruta são os produtos dos coeficientes pelos máximos da CNN ou pelos valores de TF-IDF. Decomposição e contagens.

Probabilidades calibradas no teste oficial

A calibração ajusta $\tau > 0$ para minimizar a CE em uma amostra separada. A probabilidade passa de $\sigma(\Delta)$ a $\sigma(\Delta/\tau)$, com $\Delta = z_+ - z_-$. O sinal e a classe ficam iguais.

Modelo	Acurácia	F1 macro	CE calibrada
TF-IDF + logística	90,14%	0,9014	0,2396
CNN	86,78%	0,8678	0,3037
BERT	90,94%	0,9094	0,2278

Tabela 7. Teste oficial, $n = 25.000$, com temperatura escolhida em 2.478 resenhas reservadas. A CE bruta está na Tabela 6.

Acurácia e F1 macro por tamanho de resenha

Palavras	n	TF-IDF	CNN	BERT	Qwen
Até 100	3.246	90,73 / 90,67	87,34 / 87,23	93,10 / 93,03	95,19 / 95,17
101 a 250	14.624	90,15 / 90,15	87,45 / 87,45	92,97 / 92,97	95,34 / 95,34
251 a 500	5.285	89,74 / 89,73	86,45 / 86,44	86,93 / 86,92	95,40 / 95,40
Acima de 500	1.845	90,19 / 90,16	81,52 / 81,46	82,60 / 82,46	94,36 / 94,35

Tabela 8. Cada célula indica acurácia / F1 macro, ambos em %. As faixas usam o texto inteiro, contado por espaços. Qwen é uma extensão no mesmo teste. Métricas e contagens.

Nas 1.845 resenhas acima de 500 palavras, todas recortadas na CNN, dar o texto inteiro aos mesmos pesos elevou a acurácia de 81,52% a 86,18%. Já nas duas faixas até 250 palavras, o TF-IDF ficou à frente mesmo com todas as entradas da CNN intactas. Isso delimita o que o recorte explica.

Custo computacional

Modelo	Dispositivo	Ajuste (s)	Inferência (s)
TF-IDF + logística	CPU do Apple M1	13,1	0,15
CNN	Apple M1 (MPS)	139,6	20,72
BERT	NVIDIA GB10 (CUDA)	446,4	149,91

Tabela 9. A inferência soma as quatro divisões. O ajuste do baseline inclui TF-IDF e seleção da regularização. Cada tempo vale para o dispositivo e o orçamento da receita medida.

Avaliação fora do IMDb

Em 1.000 frases por fonte, as CNNs por palavras e BPE atingiram 68,10% e 69,97% na Amazon, e 69,43% e 69,67% no Yelp [7]. No YouTube, o elogio seguido de *No one* recebeu classe positiva das seis CNNs e do Qwen, com e sem raciocínio. Escores.

Encoders e Qwen em dois protocolos

Em 2.048 prefixos, BERT, DistilBERT e ModernBERT ajustados obtiveram 88,31%, 86,80% e 91,24% na média de três sementes. CNN e TF-IDF tiveram 81,95% e 83,30%. Os ajustes completos usaram lote 32, como o BERT original. As cabeças sobre vetores congelados usaram lote 128. Métricas e receitas.

O Qwen3-8B congelado atingiu 95,26% e F1 macro de 0,9526 no teste oficial, comparando os logits de dois rótulos com até 768 tokens de resenha. Errou 635 resenhas que o TF-IDF acertou. A intervenção nas ativações é um ensaio de geração [8]. Classificação e intervenções.

5 Quando o modelo de aprendizagem profunda falha?

A mesma troca no TF-IDF e na CNN

A / leitura positiva

*I thought this movie was terrible, but it was **wonderful**.*

Termina com elogio.

B / leitura negativa

*I thought this movie was wonderful, but it was **terrible**.*

Termina com crítica.

As contagens de palavras isoladas são iguais. O TF-IDF conta o bigrama *terrible but* em A e *wonderful but* em B. O escore positivo bruto passa de 47,93% para 47,86%, com classe negativa nas duas frases. Escores do baseline.

Cada linha da figura representa um treino de CNN. Nas três sementes por palavras e nas três com BPE, a CNN deu negativa a A e positiva a B. A leitura adotada segue a oração depois de *but*.

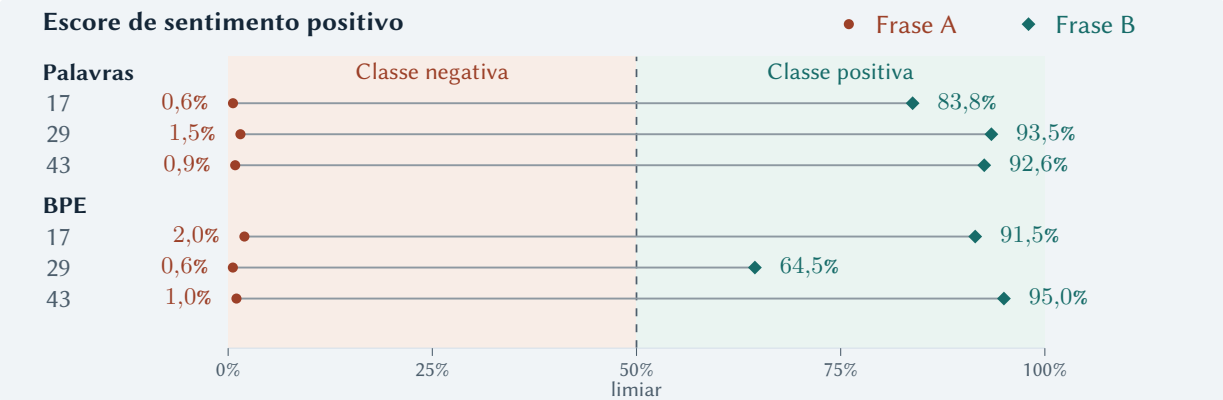


Figura 3. Escores sem calibração, por checkpoint. Da frase A para B, a média sobe de 1,0% para 89,9% com palavras, e de 1,2% para 83,7% com BPE. Escores e logits por treino. Código dos pares.

O que o filtro 89 viu

Na CNN por palavras da semente 17, o filtro 89 lê três tokens por vez. Em A, o pico está em *movie was terrible*, antes de *but*. Em B, aparece em *it was terrible*, no fim. O máximo global envia apenas a altura do pico. A camada linear combina essa altura com as dos outros 299 filtros.

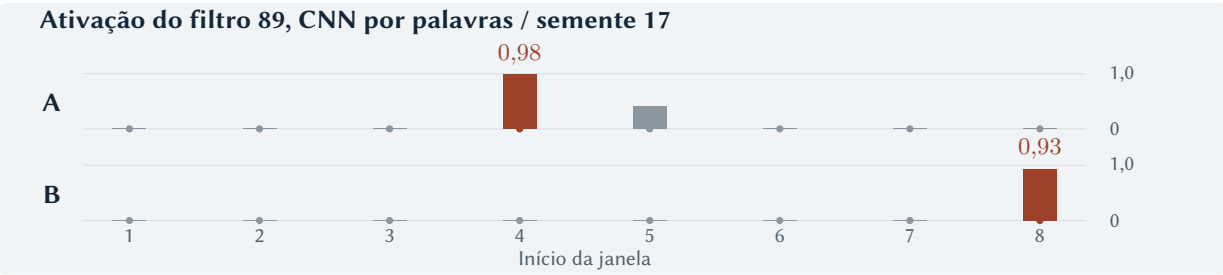


Figura 4. Ativações após a ReLU nas oito janelas de cada frase. Em A, a primeira janela *i thought this* vai de $-0{,}42$ a 0. Com passo 2, só são visitadas as janelas 1, 3, 5 e 7. O pico da janela 4 fica de fora e o máximo passa de 0,98 a 0,40. Ativações registradas.

Efeito do passo. O passo s faz a janela começar em $0, s, 2s, \ldots$. Para $L \geq k$, há $1 + \lfloor (L-k)/s \rfloor$ janelas completas. Nas mesmas 2.048 resenhas da auditoria BPE e com os pesos congelados das três CNNs por palavras, a acurácia média foi 82,00%, 81,45% e 80,97% nos passos 1, 2 e 3. O passo 2 conservou 50,2% das janelas e o passo 3, 33,6%. Resultados por passo.

Nos dois tokenizadores, a janela de cinco tokens já inclui *terrible but it was wonderful*. Sete tokens ampliariam o contexto para incluir *movie was*. A troca dos adjetivos muda as janelas. As previsões refletem a rede inteira. Para isolar o efeito de perder a posição, usamos o controle da página 8.

Controle da posição com as mesmas janelas

Montamos dois textos de controle com o elogio e a crítica em ordens opostas. Repetimos S, a frase "*I have a few thoughts about this movie.*", antes, entre e depois das opiniões para manter os vizinhos de cada uma.

A `S` `I loved the acting` `S` `I hated the ending` `S`

B `S` `I hated the ending` `S` `I loved the acting` `S`

O separador de oito palavras mantém no máximo uma opinião em cada janela. Para $k = 3, 4, 5$, A e B contêm as mesmas janelas de IDs, com as mesmas multiplicidades, nas duas tokenizações. O máximo de cada filtro descarta a posição da resposta. Assim, os 300 máximos e os dois logits coincidiram nos seis treinos. Ambas as versões expressam opiniões mistas. Código. Predições.

Teste com um pico por metade

Congelamos embeddings e filtros das três CNNs por palavras. Cada filtro entrega o máximo u da primeira metade e v da segunda. A primeira recebe a janela extra se a contagem for ímpar, e $v = 0$ se a segunda for vazia. A regressão logística recebe 600 valores. Ordenar os dois picos pelo valor descarta a metade de origem. Ao fornecer (u, v), a primeira e a segunda entrada de cada par indicam as metades. Ajustamos a regressão em 6.144 resenhas e escolhemos C por CE em 1.536 de validação. Código e grade de C.

Saída por filtro	Auditoria	Pares sintéticos	Pares editados
Máximo, cabeça original	82,00%	40,23%	40,21%
Máximo, cabeça reajustada	81,90%	39,19%	40,02%
Dois picos por valor	82,28%	39,32%	39,83%
Dois picos por posição	82,29%	39,97%	39,15%

Tabela 10. Média de três sementes. A auditoria mede acurácia por texto em 2.048 resenhas. Nas outras colunas, o par conta como acerto se as duas versões forem classificadas corretamente.

Os 256 pares sintéticos cruzam quatro moldes e dois conjuntos de oito adjetivos. Dos 488 pares editados do IMDb Contrast Set [6], excluímos 144 com entrada de IDs já vista no treino ou na validação. Oito dos 344 pares restantes ficam idênticos após o recorte, com rótulos opostos. No controle, a ordem das metades produziu margens diferentes para A e B nas três sementes, com a mesma classe. Nos pares editados, o acerto foi 39,15%, ante 39,83% com os picos ordenados pelo valor. Nos moldes sintéticos com veredito no fim, todas as cabeças acertaram 0 de 128 pares por semente. Métricas e margens por semente.

Conclusão

O TF-IDF soma sinais lexicais da resenha inteira e teve 90,14% ante 86,78% da TextCNN. Com os mesmos pesos, o texto inteiro recuperou 0,50 ponto no total e 4,66 acima de 500 palavras. A diferença persiste nos textos curtos. O controle com janelas iguais isolou a perda de posição no máximo global. No par com *but*, as janelas mudam e a decisão também depende dos filtros e da camada final. BERT ajustado (90,94%) e Qwen congelado (95,26%) superaram o baseline com outras arquiteturas, entradas e pré-treinos. Controles.

Referências

[1] Maas, A. L. et al. *Learning Word Vectors for Sentiment Analysis*. ACL, 2011. ACL Anthology.
[2] Kim, Y. *Convolutional Neural Networks for Sentence Classification*. EMNLP, 2014. ACL Anthology.
[3] Sennrich, R., Haddow, B. e Birch, A. *Neural Machine Translation of Rare Words with Subword Units*. ACL, 2016. ACL Anthology.
[4] Prince, S. J. D. *Understanding Deep Learning*. MIT Press, 2023. Caps. 6, 8, 9 e 10. Livro e materiais.
[5] Devlin, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019. ACL Anthology.
[6] Gardner, M. et al. *Evaluating Models' Local Decision Boundaries via Contrast Sets*. Findings of EMNLP, 2020. ACL Anthology.
[7] Kotzias, D. et al. *From Group to Individual Labels Using Deep Features*. KDD, 2015. Artigo. Coleção UCI.
[8] Yang, A. et al. *Qwen3 Technical Report*, 2025. arXiv.